

WORKSHOP

AI Analytics Skills for BI Analysts

Trace lineage, tune SQL, repair catalog context, and verify every AI-assisted answer before it reaches a dashboard.

FOR BI ANALYSTS · ANALYTICS ENGINEERS · DASHBOARD OWNERS

1

Find it

Tableau lineage

2

Tune it

SQL Server triage

3

Govern it

Alation remediation

4

Prove it

KPI reconciliation (optional)

This is not prompt training

A chatbot answers from what you paste. A workspace assistant reasons across your project.

We are not teaching people to prompt a chatbot. **We are teaching people to package repeatable analytics work as context-aware, tool-using, reviewable AI workflows.**

An assistant inside a BI workspace can read Tableau XML, SQL files, stored procedures, view and metric definitions, catalog docs, templates, and validation artifacts — and cite the evidence it used.

Multi-file context

Repo-aware reasoning

Tool / catalog access

Evidence-backed artifacts

Reviewable output

Chatbot vs BI workspace assistant

Chatbot

- Context = only what you paste
- You supply evidence manually
- No tools
- Output is text
- Safety is up to you

Workspace assistant

- Context = the actual repo files
- Cites files and lines it read
- Catalog / search / filesystem via MCP
- Output is reviewable artifacts
- Safety enforced by AGENTS.md + hooks

The AI analytics engineer mindset

Every lab repeats the same loop.

Open context → ask with constraints → inspect evidence
→ propose a change or artifact → validate → communicate caveats

AI is not valuable because it writes more content. **It is valuable because it helps you structure the work, inspect evidence, reduce blind spots, and create artifacts a human can review.**

Data safety and approved context

One public-safe repo for both internal and external runs.

Never include

- PII, credentials, secrets
- Production extracts
- Real server / DB / table / owner names
- Real catalog URLs or proprietary terms

Prefer instead

- Masked / invented samples
- Schemas, row counts, summaries
- Generic finance domain names
- Legacy ERP / new ERP platform (generic)

Repo tour

The repository is the durable product; the app is an optional wrapper.

repository layout

```
ai-analytics-skills/  
  AGENTS.md           # rules your assistant follows  
  labs/               # the workshop labs (start at 00-kickoff.md)  
  demos/              # sanitized demo files for each lab  
  templates/          # output templates + Mermaid standard  
  .agents/skills/     # reusable skill packages (portable)  
  .codex/              # Codex config, hooks, rules  
  evals/              # eval cases for the skills  
  docs/               # facilitator guide, data handling  
  src/                # optional React app (deck + repo browser)
```

Trace Tableau lineage

What does this dashboard depend on?



Primary question

What does this Tableau dashboard depend on?



Deliverable

Lineage doc + Mermaid diagram + source inventory



Key discipline

Separate confirmed from inferred; unknowns become owner questions.

prompt pattern

Trace each custom SQL block to its source objects. Return:

1. Data sources
2. Custom SQL blocks
3. Referenced views / procs
4. Upstream tables / linked servers
5. Confirmed vs inferred lineage
6. Unknowns for owner review
7. Mermaid data-flow diagram

Triage SQL Server performance safely

Why is this slow, and what is the smallest safe fix?



Guardrails

Don't change logic or grain; don't add indexes; validate rows and totals.



Phase 1 — diagnose

Explain risks; do not rewrite yet. Collect baseline evidence.



Phase 2 — rewrite

Smallest safe change + a validation query that proves equivalence.

two-phase prompt

```
// Phase 1: diagnose
1. What the query does
2. Likely performance risks
3. Safe-for-analyst fixes
4. DBA / data-eng review items
5. Baseline to collect first

// Phase 2: smallest safe rewrite
Preserve grain, rows, totals.
Include a validation query.
```


Repair catalog context with AI

The full agentic stack — with a safe-write contract.



The stack

AGENTS.md + skill + MCP + hooks + rules
(read free, writes gated).



Safe-write contract

Read-only, then draft a local diff, human review,
then write.



Non-negotiable

The local diff is the dry run — no write without
cited evidence.

safe-write contract

1. Read catalog + SQL read-only
2. Draft the change to a
local diff (outputs/)
3. Human reviews vs evidence
4. Only then call the write tool

```
allow:    read-only search
approval: catalog writes
block:    destructive ops
```

Prove the KPI still reconciles

Did the metric survive a source / platform change?

λ

LLM drafts the spec

An enterprise LLM API proposes a structured validation spec.

π

Python scores it

Deterministic checks: same inputs, same verdict — every time.

=

Determinism contract

Thresholds live in the spec; the LLM never decides pass/fail.

outputs/

```
validation_spec.json
reconciliation_results.csv
failed_checks.csv
validation_report.md
catalog_doc_patch.md

# borrow: dbt-audit-helper,
# Great Expectations, dbt-codegen
```

Verification checklist

The analyst remains accountable for the claim.

Metric

Accurate to the business?

Data

Right grain and scope?

SQL

Joins fan out? Filters applied?

Story

Supported strictly by evidence?

the reviewer prompt

Before I use this answer, list every assumption, source file, query, unresolved risk, and manual check. Separate what you confirmed from what you inferred.

CONTINUE

Make it a habit

Copy the skills into your own repo, add a root AGENTS.md, keep writes gated behind approval, and run the verification checklist on real work.

Adopt the loop

Reuse the skills

Safe by default

Eval in CI

`docs/next-steps.md` · `labs/06-verification-checklist.md`